

2026《Linux内核原理与分析》第2周作业

作者：叶丁再

学号：20262809

课程：《Linux内核原理与分析》

课程资料来源：《Linux内核分析》MOOC：[课程页面](#)

转载说明：原创内容如被转载，请注明原文出处。

一、实验目的

C 程序中的函数调用看起来很简单：传入参数、执行函数、返回结果。但处理器执行时，还要保存返回位置、维护当前函数的工作状态，并在函数结束后恢复调用者的状态。

本实验使用 GCC 将一个简单的 C 程序生成 32 位汇编文件，再结合函数调用分析参数、返回地址和栈帧的变化。通过这条从 C 代码到汇编指令的路径，我希望进一步理解计算机怎样执行程序。

二、程序结构与计算关系

课程题面给出的参考程序如下。题目要求自行修改数字；终端截图没有展示 main.c 的内容。所以下面代码用于说明题目中的函数关系，发布前应替换为本次实验实际使用的 main.c，并根据实际数字核对结果。

```
int g(int x)
{
    return x + 3;
}

int f(int x)
{
    return g(x);
}

int main(void)
{
    return f(8) + 1;
}
```

对上面的参考程序，g 函数将参数加 3，f 函数调用 g 并返回它的结果，main 函数调用 f 后再加 1。因此参考示例的计算结果为：

$f(8) = 8 + 3 = 11$ ；main 返回 $11 + 1 = 12$ 。

如果将本人程序中的三个数分别记作 C、N、M，那么整体关系为： $g(x) = x + C$ ， $f(x) = g(x)$ ，main 返回 $N + C + M$ 。函数调用的层次不随这些数字变化，但实际结果应以本人 main.c 中的数字为准。

这段程序没有调用 printf，因此即使运行成功，也不会自动在终端打印返回值。main 返回的整数是进程退出状态，不等同于屏幕输出。

三、生成汇编文件

我在 Linux 实验桌面进入保存 main.c 的目录后，执行了以下命令：

```
gcc -S -o main.s main.c -m32
```

参数含义如下：

- -S：编译到汇编文本后停止，不继续生成目标文件或可执行文件；
- -o main.s：把输出文件命名为 main.s；
- -m32：要求 GCC 生成 32 位目标代码。

截图记录了第一次尝试时 zsh 报出 command not found: \$；随后重新输入 gcc 命令，终端返回提示符。再次查看目录时，列表中出现了 main.s。这可以证明汇编文件已经生成。

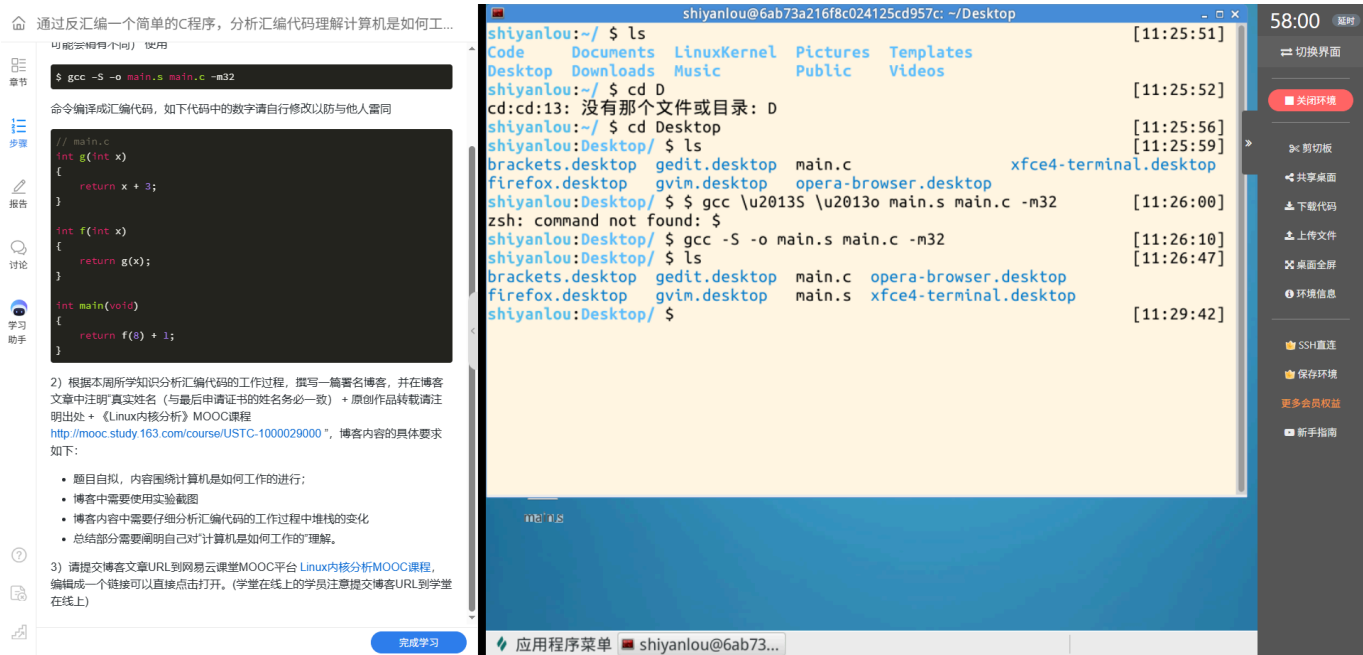


图 1：终端中的编译命令及目录列表。截图显示 main.c 和生成后的 main.s；截图没有显示 main.s 的内容，也没有显示 GDB 中的寄存器或栈内存。

四、C 函数如何映射到汇编操作

GCC 将 C 代码转换为目标处理器的指令。-m32 指定 32 位目标。GCC 常见的 AT&T 汇编语法中，寄存器名前百分号，立即数前有美元符号，指令操作数通常按“源、目标”的顺序书写。编译器版本、目标环境和优化选项不同，最终生成的指令排列可能不同。

虽然这次截图没有展示 main.s 的具体内容，但可以从程序的调用关系说明其中需要完成的工作：

C 代码中的动作	汇编需要完成的工作	对程序状态的影响
main 调用 f	准备参数并执行 call	保存返回到 main 的位置，转入 f
f 调用 g	把参数传给 g 并执行 call	保存返回到 f 的位置，进入下一层调用
g 计算 x + C	读取参数并执行加法	计算结果通常放在 EAX 中
g、f 返回	执行 ret	取出返回地址，恢复到调用者继续执行

C 代码中的动作	汇编需要完成的工作	对程序状态的影响
main 加上 M	对返回结果执行加法	得到 main 最终返回的整数

在汇编中，push 通常用于压入数据，call 用于调用函数，mov 用于传送数据，add 用于加法，ret 用于返回。上述是对指令职责的解释；具体使用哪些寄存器、是否出现单独的 push 或 add 指令，应以实际打开的 main.s 为准。

五、函数调用过程中栈的变化

以下按常见的 32 位 Linux x86 cdecl 调用约定说明。32 位栈向较低内存地址增长，ESP 指向当前栈顶。一次 32 位 push 或 call 通常会让 ESP 减少 4 字节，并在新的栈顶位置保存一个 4 字节值。

1. main 准备参数并调用 f

main 准备参数 N 后调用 f。常见的 32 位 cdecl 方式会先把参数放到栈上。执行 call f 时，处理器把 call 后一条指令的地址压入栈中，作为返回地址，然后把控制流转到 f。

刚进入 f 时，栈顶附近包含返回地址和参数。返回地址使 f 完成后能够回到 main 中正确的位置；参数则让 f 能取得调用者传入的整数。

2. f 建立栈帧并调用 g

采用帧指针的典型函数序言会先保存调用者的 EBP，再将当前 ESP 复制到 EBP，作为当前函数栈帧的基准。建立栈帧后，参数和返回地址通常位于以下位置：

```

较高内存地址
[EBP + 8]  第一个参数 x
[EBP + 4]  返回到调用者的地址
[EBP]      调用者保存的 EBP
[EBP - ...] 局部变量或临时空间（如果有）
较低内存地址
ESP 附近为当前栈顶

```

f 读取自己的参数，再把它传给 g 并执行 call g。这个调用又保存一个返回地址，因此在 g 执行期间，栈上同时保留着回到 f 和回到 main 所需的信息。函数调用嵌套越深，尚未结束的调用状态就越多。

3. g 计算结果并返回

g 从参数位置读出 x，计算 $x + C$ 。按照常见的 32 位整数返回约定，结果放入 EAX。函数恢复自己的栈帧后执行 ret；ret 从栈顶取出返回地址，程序回到 f 中调用 g 之后的位置。

以题面参考数字为例，g 计算 $8 + 3$ ，返回 11。若本人程序使用了不同数字，应将这里的计算替换成实际结果。

4. f 返回给 main，main 完成计算

f 得到 g 返回的结果后，将它传回 main。f 的栈帧恢复，ret 使控制流回到 main。cdecl 通常由调用方清理参数占用的栈空间。之后 main 对返回结果加上常数 M，并将最终值返回给程序启动代码。

因此，一次嵌套调用中的栈变化可以概括为：

1. 调用方准备参数；
2. call 保存返回地址并跳到被调用函数；
3. 被调用函数保存旧帧指针，并为当前栈帧设置基准；
4. 被调用函数可以继续压入参数并调用下一层函数；
5. 函数返回时恢复栈帧，ret 取出返回地址；
6. 控制流回到调用者，调用者继续计算并清理参数空间。

实际编译器可能省略帧指针、改变栈空间分配方式或重排指令。因此，EBP 相对偏移和上面的栈帧图是常见布局，需与本次 main.s 中的函数序言、参数读取和函数返回指令相互核对。

六、从这次实验理解计算机如何工作

我对“计算机如何工作”的理解是：C 代码描述程序要完成的操作，编译器把这些操作转换为处理器能够执行的指令；处理器依据指令地址取指、译码和执行，使用寄存器保存当前计算结果和控制流状态，并使用内存保存程序数据。

函数调用把这些部分联系起来：call 改变控制流并保存返回位置，栈保留参数和函数现场，函数将结果放入返回寄存器后再通过 ret 回到调用者。原来一句简单的 C 函数调用，在底层包含了状态保存、地址跳转、数据计算和状态恢复等一系列操作。理解这些过程，能帮助我从源代码继续追踪到机器指令和运行时数据。

七、实验记录与后续核对

本次实验截图证明了 GCC 命令执行成功以及 main.s 已生成。当前截图没有展示汇编文本，也没有运行时 ESP、EBP 和栈内存的读数。因此，本文对具体栈帧的说明是依据常见 32 位调用约定进行的分析，不应写成已经通过 GDB 逐条观察到的结果。

发布前请将参考代码和示例数字改为本人 main.c 中的实际内容，并检查计算结果与汇编分析是否一致。若补充了 main.s 或 GDB 截图，可放入图片目录后在对应段落引用。

参考资料

- 《Linux内核原理与分析》课程实验要求。
- 《Linux内核分析》MOOC：[课程页面](#)。
- GCC 编译命令及本次实验终端截图。